

Hotel Management System

A full-stack REST application for managing hotels,
rooms, bookings and users — built with Spring Boot & React

Spring Boot 3

React + Vite

MySQL

JWT Auth

REST API

Team :

Yashasree Pola

Mohan Sai Dhanekula

Mallikarjuna Urukadle

Hemanthvarma Addepallibalaraju

VenkataSakethkumar Tallam

Agenda

01 Project Overview
What is this system and why was it built?

03 System Architecture
How the layers connect — end to end

05 API Design
REST endpoints, JWT security and RBAC

07 Testing & Docs
JUnit, Mockito, integration tests, Swagger

09 Challenges & Learnings
What I solved and what I learned

02 Tech Stack
Technologies and tools used across all layers

04 Modules & Features
Hotels, Rooms, Bookings, Users and Auth

06 Database Design
Entities, relationships and ER diagram

08 Live Demo Highlights
Key screens and Swagger UI walkthrough

10 Thank You
Q & A

Project Overview

What is this system?

The Problem

- Hotels managed via spreadsheets or paper
- No real-time room availability checking
- Manual booking — prone to double-booking
- No role separation between staff and guests

5

Core Modules

20+

REST Endpoints

The Solution

- Centralised web-based management platform
- Live availability engine with overlap detection
- Automated booking with price calculation
- JWT-secured RBAC for Admin and Customer

2

User Roles

100%

REST API

Tech Stack

Every layer, every tool

Frontend

- › React 19 + Vite
- › React Router DOM 7
- › Axios
- › Tailwind CSS 4

Backend

- › Java 17
- › Spring Boot 3.2.5
- › Spring Security 6
- › Spring Data JPA
- › JWT (jjwt 0.11.5)
- › Lombok

Database

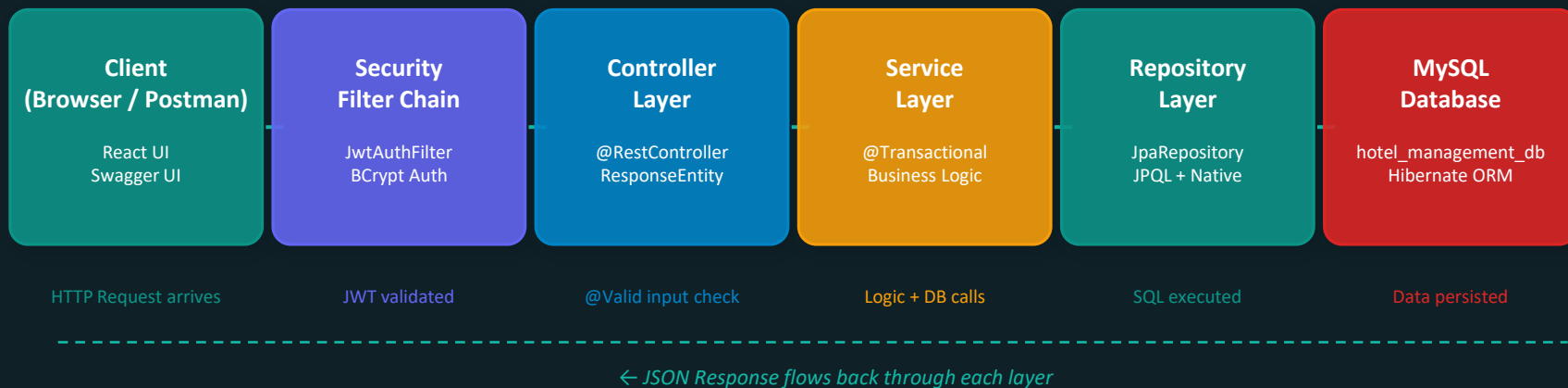
- › MySQL 8
- › Hibernate ORM 6
- › JPA Entities
- › Auto DDL (update)

Dev Tools

- › IntelliJ IDEA
- › Postman
- › Swagger / OpenAPI
- › JUnit 5 + Mockito
- › Maven 3
- › Git

System Architecture

Layered architecture — each layer has one responsibility



Modules & Features

Five core modules powering the entire system

01

Hotels

- Create / Read / Update / Delete hotels
- Upload & serve hotel images
- Keyword search (name or city)
- Paginated listing with sorting

02

Rooms

- Add rooms to any hotel
- Types: Single, Double, Suite, Deluxe
- Real-time availability check
- Price stored as BigDecimal (no rounding)

Modules & Features

Four core modules powering the entire system

03

Bookings

- Date overlap prevention engine
- Auto price calculation (nights × rate)
- Pay-later with advance amount
- Cancel booking workflow

4 & 5

Auth & Users

- JWT-based stateless authentication
- BCrypt password hashing
- ADMIN vs CUSTOMER roles (RBAC)
- Register, Login, protected routes

User Roles & Permissions

Role-Based Access Control (RBAC) enforced at the API level

ADMIN

- ✓ Create, edit, delete Hotels
- ✓ Add, update, delete Rooms
- ✓ View ALL bookings in the system
- ✓ Upload hotel images
- ✓ Manage all user accounts
- ✓ Access admin dashboard
- ✓ Full access to every endpoint

CUSTOMER

- ✓ Browse all hotels (public)
- ✓ View room availability by date
- ✓ Create bookings (with price calc)
- ✓ Pay-later / advance option
- ✓ View & cancel own bookings
- ✗ Cannot access admin features
- ✗ Cannot view other users' data



New registrations always receive CUSTOMER role · ADMIN is assigned manually via database

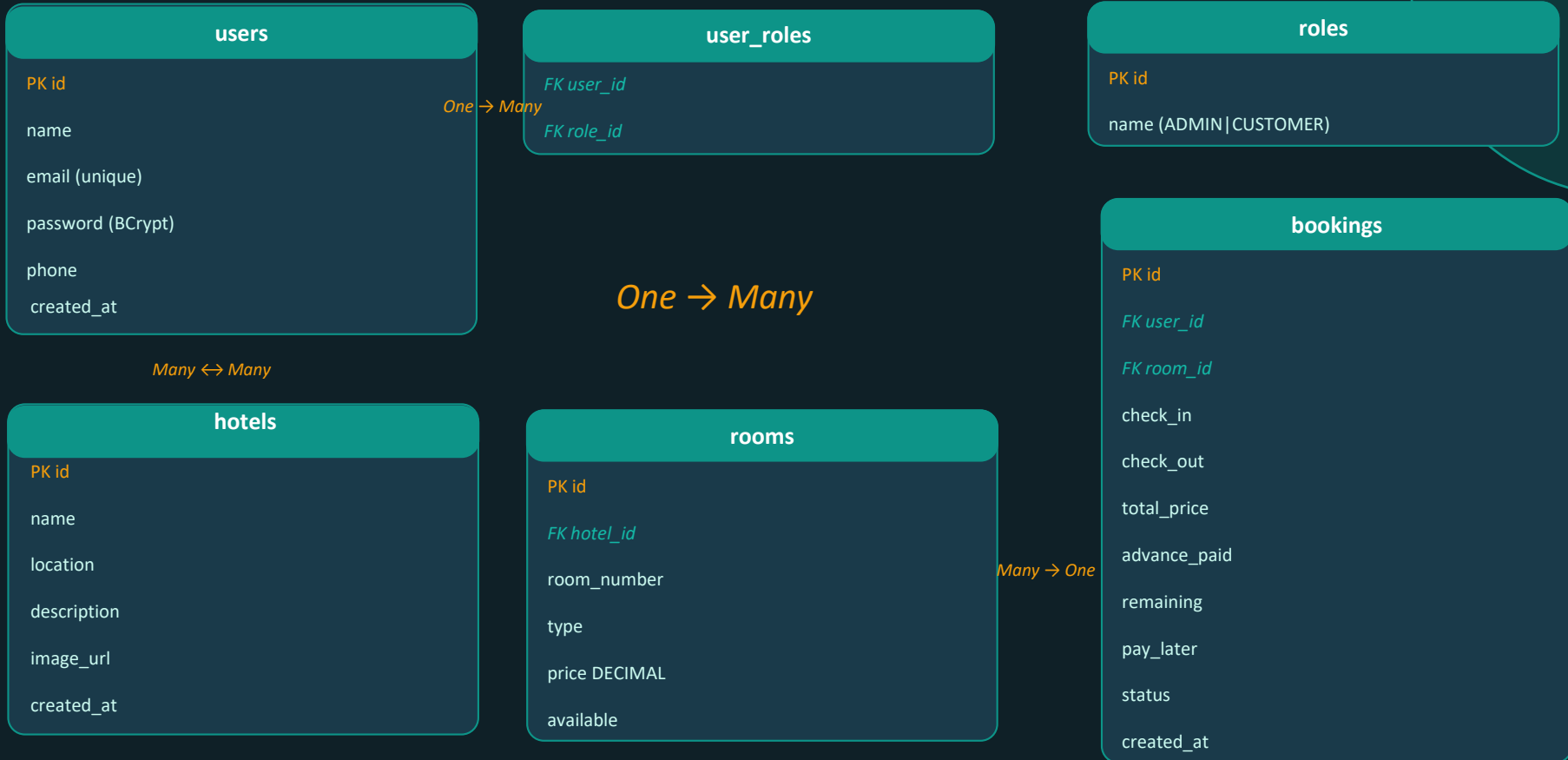
API Design

RESTful endpoints · JWT Bearer authentication · Standard HTTP status codes

Method	Endpoint	What it does	Auth
POST	/api/auth/register	Register new user	Public
POST	/api/auth/login	Login → receive JWT	Public
GET	/api/hotels	All hotels (paginated + sortable)	Public
POST	/api/hotels	Create hotel	ADMIN
PUT	/api/hotels/{id}	Update hotel	ADMIN
DELETE	/api/hotels/{id}	Delete hotel	ADMIN
GET	/api/hotels/search?keyword=	Search hotels	Public
GET	/api/rooms/available	Rooms free for given dates	Public
POST	/api/rooms	Add room to hotel	ADMIN
POST	/api/bookings	Create booking	Customer
GET	/api/bookings/my	My bookings (paginated)	Customer
PUT	/api/bookings/{id}/cancel	Cancel booking	Customer
GET	/api/bookings	All bookings (paginated)	ADMIN

Database Design

5 tables · Normalised schema · JPA manages DDL automatically



Security & JWT Flow

Stateless authentication — the server stores no session



LOGIN FLOW

1. User sends email + password
2. AuthenticationManager verifies
3. BCrypt.matches() checks hash
4. JWT generated (email + roles + expiry)
5. Token returned to client (24h valid)

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWJSM...

Header (algo) · Payload (email, roles, exp) · Signature



EVERY PROTECTED REQUEST

1. Client adds Authorization: Bearer <token>
2. JwtAuthFilter intercepts the request
3. Token signature + expiry validated
4. User identity loaded from DB
5. SecurityContextHolder updated
6. SecurityConfig checks required role
7. Request reaches controller (or 403)

Testing & Documentation

JUnit 5 · Mockito · MockMvc · Springdoc OpenAPI / Swagger UI

Unit Tests

@ExtendWith(MockitoExtension)
@Mock — fake dependencies
@InjectMocks — real service
when().thenReturn() — stubs
verify() — assert calls

Tests covered:

HotelService — 5 tests
BookingService — 3 tests

Integration Tests

@SpringBootTest — full context
@AutoConfigureMockMvc
MockMvc — simulated HTTP
andExpect(status().is201())
andExpect(jsonPath("\$.email"))

Tests covered:

POST /register → 201
POST /register (bad email) → 400

Swagger UI

springdoc-openapi 2.5.0
/swagger-ui/index.html
@Tag — group endpoints
@Operation — descriptions
Authorize button (JWT)

Features:

Try endpoints in browser
Auto-generated schemas



All tests pass — 9 tests total · 0 failures · 0 errors · context loads cleanly

Live Demo Highlights

Key flows to demonstrate during the live walkthrough

01 Register & Login

POST /api/auth/register → 201
POST /api/auth/login → JWT token
Show roles: [ADMIN, CUSTOMER] in response

02 Hotel & Room CRUD

POST /api/hotels → 201 Created
POST /api/rooms with hotelId
GET
/api/hotels?page=0&size=5&sortBy=name

03 Booking Engine

GET
/api/rooms/available?hotelId=1&checkIn=
...
POST /api/bookings → auto price calc
totalPrice = nights × room price

04 Security (RBAC)

Try POST /api/hotels with customer token
→ 403 Forbidden (proves RBAC works)
Logout → 403 on any protected call

05 Pagination & Sort

GET /api/hotels?page=0&size=1 → page 1 of N
GET /api/hotels?page=1&size=1 → page 2 of N
Frontend: prev/next buttons + sort dropdown

06 Swagger UI

Open swagger-ui/index.html
Click Authorize → paste JWT token
Try every endpoint from the browser UI

Challenges & Learnings

Real problems solved — real skills gained

⚡ Java 25 / Lombok Incompatibility

Diagnosed `ExceptionInInitializerError`, downgraded to Java 17 LTS — now the standard for Spring Boot 3.

⚡ JWT Role Mismatch After DB Change

Roles in JWT are frozen at login time. Discovered this by changing a role in DB and seeing stale token. Fix: re-login.

⚡ Booking Date Overlap Logic

Wrote a native SQL NOT IN subquery: `check_in < new.checkOut AND check_out > new.checkIn`. Prevents double-booking.

⚡ CSRF vs Stateless APIs

Disabled CSRF in `SecurityConfig` — CSRF protects session-based apps. JWT is stateless, so CSRF is irrelevant.

⚡ LazyInitializationException

Used JOIN FETCH in JPQL for `UserRepository` to load roles in one query. `@Transactional` scope keeps session open.

Key Takeaways

What this project demonstrates



Layered Architecture

Controller → Service → Repository — every layer has one job, tested independently.



Stateless Security

JWT tokens make the API horizontally scalable. No server-side sessions. BCrypt protects every password.



SOLID Principles

Service interfaces, constructor injection, single-responsibility DTOs — not just theory.



RBAC in Practice

URL-level role guards in SecurityConfig + method-level checks. 403 proven by live negative test.



Auto Documentation

Swagger UI + Springdoc — any developer can explore and test every endpoint with zero setup.



Verified with Tests

Unit tests (Mockito) + integration tests (MockMvc) — code confidence, not just hope.

Thank You

Hotel Management System

Questions & Answers

Spring Boot 3

React + Vite

MySQL

JWT

Swagger

JUnit 5